VELORA — Module Tenant (Part 1)

Migrations · Models · Enums · Traits · Repository

Folder Structure

```
app/Modules/Tenant/
├── Controllers/
│   ├── BusinessController.php
│   └── BranchController.php
├── DTOs/
│   ├── CreateBusinessDTO.php
│   ├── UpdateBusinessDTO.php
│   ├── CreateBranchDTO.php
│   └── UpdateBranchDTO.php
├── Enums/
│   ├── BusinessStatus.php
│   └── BranchStatus.php
├── Events/
│   └── BusinessCreated.php
├── Exceptions/
│   ├── BusinessNotFoundException.php
│   └── BranchNotFoundException.php
├── Models/
│   ├── Business.php
│   └── Branch.php
├── Policies/
│   ├── BusinessPolicy.php
│   └── BranchPolicy.php
├── Repositories/
│   ├── Contracts/
│   │   ├── BusinessRepositoryInterface.php
│   │   └── BranchRepositoryInterface.php
│   └── Eloquent/
│       ├── BusinessRepository.php
│       └── BranchRepository.php
├── Requests/
│   ├── StoreBusinessRequest.php
│   ├── UpdateBusinessRequest.php
│   ├── StoreBranchRequest.php
│   └── UpdateBranchRequest.php
├── Resources/
│   ├── BusinessResource.php
│   └── BranchResource.php
├── Services/
│   ├── BusinessService.php
│   └── BranchService.php
└── Routes/
    └── api.php
```

1. Migrations

Businesses Table

```php
<?php
// database/migrations/tenant/2026_05_19_000001_create_businesses_table.php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('businesses', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->uuid('owner_id');

            $table->string('name', 150);
            $table->string('slug', 160)->unique();
            $table->string('email', 150)->unique();
            $table->string('phone', 20)->nullable();
            $table->text('address')->nullable();
            $table->string('city', 100)->nullable();
            $table->string('province', 100)->nullable();
            $table->string('postal_code', 10)->nullable();
            $table->string('country', 2)->default('ID');

            $table->string('logo')->nullable();
            $table->string('currency', 3)->default('IDR');
            $table->string('timezone', 50)->default('Asia/Jakarta');
            $table->string('language', 5)->default('id');

            $table->string('tax_id', 30)->nullable()->comment('NPWP');
            $table->string('status', 20)->default('active')->index();

            $table->json('settings')->nullable()->comment('Business-level config override');
            $table->timestamp('verified_at')->nullable();
            $table->timestamps();
            $table->softDeletes();

            $table->foreign('owner_id')->references('id')->on('users');
            $table->index(['status', 'created_at']);
        });
    }

    public function down(): void
```

```
    {
        Schema::dropIfExists('businesses');
    }
};
```

Branches Table

```php
<?php
// database/migrations/tenant/2026_05_19_000002_create_branches_table.php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('branches', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->uuid('business_id');

            $table->string('name', 100);
            $table->string('code', 20)->nullable()->comment('Branch code, e.g. BR-001');
            $table->text('address')->nullable();
            $table->string('city', 100)->nullable();
            $table->string('phone', 20)->nullable();
            $table->string('email', 150)->nullable();
            $table->boolean('is_main')->default(false)->index();
            $table->string('status', 20)->default('active')->index();
            $table->json('settings')->nullable();

            $table->timestamps();
            $table->softDeletes();

            $table->foreign('business_id')
                  ->references('id')->on('businesses')
                  ->cascadeOnDelete();

            $table->unique(['business_id', 'code']);
            $table->index(['business_id', 'status']);
            $table->index(['business_id', 'is_main']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('branches');
    }
};
```

Add business_id & branch_id to users

```php
<?php
// database/migrations/tenant/2026_05_19_000003_add_tenant_columns_to_users_table.php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            // business_id sudah ada di migration users, ini hanya untuk branch_id
            $table->uuid('branch_id')->nullable()->after('business_id');

            $table->foreign('business_id')->references('id')->on('businesses')->nullOnDelete();
            $table->foreign('branch_id')->references('id')->on('branches')->nullOnDelete();

            $table->index(['business_id', 'branch_id']);
        });
    }

    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropForeign(['business_id']);
            $table->dropForeign(['branch_id']);
            $table->dropColumn(['branch_id']);
        });
    }
};
```

2. Enums

app/Modules/Tenant/Enums/BusinessStatus.php

```php
<?php

namespace App\Modules\Tenant\Enums;

enum BusinessStatus: string
{
    case Active   = 'active';
    case Inactive = 'inactive';
    case Suspended = 'suspended';
    case Pending   = 'pending';

    public function label(): string
    {
        return match ($this) {
            self::Active    => 'Aktif',
            self::Inactive  => 'Tidak Aktif',
            self::Suspended => 'Ditangguhkan',
            self::Pending   => 'Menunggu Verifikasi',
        };
    }

    public function isOperational(): bool
    {
        return $this === self::Active;
    }
}
```

app/Modules/Tenant/Enums/BranchStatus.php

```php
<?php

namespace App\Modules\Tenant\Enums;

enum BranchStatus: string
{
    case Active   = 'active';
    case Inactive = 'inactive';
    case Closed   = 'closed';

    public function label(): string
    {
        return match ($this) {
            self::Active   => 'Aktif',
            self::Inactive => 'Tidak Aktif',
            self::Closed   => 'Tutup',
        };
    }
}
```

3. Models

app/Modules/Tenant/Models/Business.php

```php
<?php

namespace App\Modules\Tenant\Models;

use App\Models\User;
use App\Modules\Tenant\Enums\BusinessStatus;
use App\Shared\Traits\HasUuid;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasMany;
use Illuminate\Database\Eloquent\SoftDeletes;
use Illuminate\Support\Str;

class Business extends Model
{
    use HasFactory, HasUuid, SoftDeletes;

    protected $fillable = [
        'id', 'owner_id', 'name', 'slug', 'email', 'phone',
        'address', 'city', 'province', 'postal_code', 'country',
        'logo', 'currency', 'timezone', 'language',
        'tax_id', 'status', 'settings', 'verified_at',
    ];

    protected function casts(): array
    {
        return [
            'status'      => BusinessStatus::class,
            'settings'    => 'array',
            'verified_at' => 'datetime',
        ];
    }

    /* ── Boot: auto-generate slug ── */

    protected static function boot(): void
    {
        parent::boot();

        static::creating(function (self $model) {
            if (empty($model->slug)) {
                $model->slug = static::generateUniqueSlug($model->name);
            }
```

```
        });
    }

    private static function generateUniqueSlug(string $name): string
    {
        $slug  = Str::slug($name);
        $count = static::where('slug', 'like', "{$slug}%")->count();
        return $count ? "{$slug}-{$count}" : $slug;
    }

    /* ── Relations ── */

    public function owner(): BelongsTo
    {
        return $this->belongsTo(User::class, 'owner_id');
    }

    public function users(): HasMany
    {
        return $this->hasMany(User::class, 'business_id');
    }

    public function branches(): HasMany
    {
        return $this->hasMany(Branch::class, 'business_id');
    }

    public function mainBranch(): \Illuminate\Database\Eloquent\Relations\HasOne
    {
        return $this->hasOne(Branch::class, 'business_id')->where('is_main', true);
    }

    /* ── Helpers ── */

    public function isActive(): bool
    {
        return $this->status === BusinessStatus::Active;
    }

    public function hasActiveSubscription(): bool
    {
        // Akan diimplementasikan saat Module Subscription selesai
        // Sementara return true agar tidak memblokir development
        return true;
    }

    public function getLogoUrlAttribute(): ?string
    {
        return $this->logo
```

```
            ? \Illuminate\Support\Facades\Storage::url($this->logo)
            : null;
    }

    public function getSetting(string $key, mixed $default = null): mixed
    {
        return data_get($this->settings, $key, $default);
    }
}
```

app/Modules/Tenant/Models/Branch.php

```php
<?php

namespace App\Modules\Tenant\Models;

use App\Modules\Tenant\Enums\BranchStatus;
use App\Shared\Traits\HasUuid;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\SoftDeletes;

class Branch extends Model
{
    use HasFactory, HasUuid, SoftDeletes;

    protected $fillable = [
        'id', 'business_id', 'name', 'code',
        'address', 'city', 'phone', 'email',
        'is_main', 'status', 'settings',
    ];

    protected function casts(): array
    {
        return [
            'status'  => BranchStatus::class,
            'is_main' => 'boolean',
            'settings'=> 'array',
        ];
    }

    /* ── Relations ── */

    public function business(): BelongsTo
    {
        return $this->belongsTo(Business::class, 'business_id');
    }

    public function users(): \Illuminate\Database\Eloquent\Relations\HasMany
    {
        return $this->hasMany(\App\Models\User::class, 'branch_id');
    }

    /* ── Helpers ── */

    public function isActive(): bool
    {
        return $this->status === BranchStatus::Active;
```

```
    }
}
```

4. Repository Interfaces

app/Modules/Tenant/Repositories/Contracts/BusinessRepositoryInterface.php

```php
<?php

namespace App\Modules\Tenant\Repositories\Contracts;

use App\Modules\Tenant\Models\Business;
use Illuminate\Pagination\LengthAwarePaginator;

interface BusinessRepositoryInterface
{
    public function findById(string $id): ?Business;
    public function findBySlug(string $slug): ?Business;
    public function findByOwner(string $ownerId): ?Business;
    public function all(array $filters = []): LengthAwarePaginator;
    public function create(array $data): Business;
    public function update(string $id, array $data): Business;
    public function delete(string $id): bool;
}
```

app/Modules/Tenant/Repositories/Contracts/BranchRepositoryInterface.php

```php
<?php

namespace App\Modules\Tenant\Repositories\Contracts;

use App\Modules\Tenant\Models\Branch;
use Illuminate\Database\Eloquent\Collection;
use Illuminate\Pagination\LengthAwarePaginator;

interface BranchRepositoryInterface
{
    public function findById(string $id): ?Branch;
    public function findByBusiness(string $businessId, array $filters = []): LengthAwarePaginator;
    public function activeByBusiness(string $businessId): Collection;
    public function countByBusiness(string $businessId): int;
    public function create(array $data): Branch;
    public function update(string $id, array $data): Branch;
    public function delete(string $id): bool;
    public function setMainBranch(string $branchId, string $businessId): void;
}
```

5. Repository Implementations

app/Modules/Tenant/Repositories/Eloquent/BusinessRepository.php

```php
<?php

namespace App\Modules\Tenant\Repositories\Eloquent;

use App\Modules\Tenant\Models\Business;
use App\Modules\Tenant\Repositories\Contracts\BusinessRepositoryInterface;
use App\Shared\Repositories\BaseRepository;
use Illuminate\Pagination\LengthAwarePaginator;

class BusinessRepository extends BaseRepository implements BusinessRepositoryInterface
{
    public function __construct(Business $model)
    {
        parent::__construct($model);
    }

    public function findById(string $id): ?Business
    {
        return Business::with(['owner', 'mainBranch'])->find($id);
    }

    public function findBySlug(string $slug): ?Business
    {
        return Business::where('slug', $slug)->with(['owner', 'mainBranch'])->first();
    }

    public function findByOwner(string $ownerId): ?Business
    {
        return Business::where('owner_id', $ownerId)->with(['mainBranch'])->first();
    }

    public function all(array $filters = []): LengthAwarePaginator
    {
        return Business::query()
            ->when($filters['status'] ?? null, fn($q, $v) => $q->where('status', $v))
            ->when($filters['search'] ?? null, fn($q, $v) => $q->where('name', 'like', "%{$v}%"))
            ->with(['owner', 'mainBranch'])
            ->latest()
            ->paginate($filters['per_page'] ?? 15);
    }

    public function create(array $data): Business
    {
        return Business::create($data);
```

```
    }

    public function update(string $id, array $data): Business
    {
        $business = Business::findOrFail($id);
        $business->update($data);
        return $business->fresh(['owner', 'mainBranch']);
    }

    public function delete(string $id): bool
    {
        return Business::findOrFail($id)->delete();
    }
}
```

app/Modules/Tenant/Repositories/Eloquent/BranchRepository.php

```php
<?php

namespace App\Modules\Tenant\Repositories\Eloquent;

use App\Modules\Tenant\Models\Branch;
use App\Modules\Tenant\Repositories\Contracts\BranchRepositoryInterface;
use App\Shared\Repositories\BaseRepository;
use Illuminate\Database\Eloquent\Collection;
use Illuminate\Pagination\LengthAwarePaginator;
use Illuminate\Support\Facades\DB;

class BranchRepository extends BaseRepository implements BranchRepositoryInterface
{
    public function __construct(Branch $model)
    {
        parent::__construct($model);
    }

    public function findById(string $id): ?Branch
    {
        return Branch::with('business')->find($id);
    }

    public function findByBusiness(string $businessId, array $filters = []): LengthAwarePaginator
    {
        return Branch::where('business_id', $businessId)
            ->when($filters['status'] ?? null, fn($q, $v) => $q->where('status', $v))
            ->when($filters['search'] ?? null, fn($q, $v) => $q->where('name', 'like', "%{$v}%"))
            ->latest()
            ->paginate($filters['per_page'] ?? 15);
    }

    public function activeByBusiness(string $businessId): Collection
    {
        return Branch::where('business_id', $businessId)
            ->where('status', 'active')
            ->orderBy('is_main', 'desc')
            ->get();
    }

    public function countByBusiness(string $businessId): int
    {
        return Branch::where('business_id', $businessId)->count();
    }

    public function create(array $data): Branch
    {
```

```php
        return Branch::create($data);
    }

    public function update(string $id, array $data): Branch
    {
        $branch = Branch::findOrFail($id);
        $branch->update($data);
        return $branch->fresh();
    }

    public function delete(string $id): bool
    {
        return Branch::findOrFail($id)->delete();
    }

    public function setMainBranch(string $branchId, string $businessId): void
    {
        DB::transaction(function () use ($branchId, $businessId) {
            // Lepas main dari semua branch di business ini
            Branch::where('business_id', $businessId)->update(['is_main' => false]);
            // Set branch yg dipilih jadi main
            Branch::where('id', $branchId)->update(['is_main' => true]);
        });
    }
}
```

6. Register Repository Bindings

app/Providers/RepositoryServiceProvider.php

```php
<?php

namespace App\Providers;

use Illuminate\Support\ServiceProvider;

// Tenant
use App\Modules\Tenant\Repositories\Contracts\BusinessRepositoryInterface;
use App\Modules\Tenant\Repositories\Eloquent\BusinessRepository;
use App\Modules\Tenant\Repositories\Contracts\BranchRepositoryInterface;
use App\Modules\Tenant\Repositories\Eloquent\BranchRepository;

class RepositoryServiceProvider extends ServiceProvider
{
    public function register(): void
    {
        $this->app->bind(BusinessRepositoryInterface::class, BusinessRepository::class);
        $this->app->bind(BranchRepositoryInterface::class, BranchRepository::class);
    }
}
```